

Manasight+ Learning Guide

Who This Is For

This guide is for someone who is smart, curious, and new to programming.

If you are comfortable reading complicated ideas but do not yet feel fluent in code, this document is for you. I wrote it in plain English on purpose. The goal is not just to explain what Manasight+ does, but to use Manasight+ as a way to teach the basic ideas that show up in real software projects.

What This Project Does In One Sentence

Manasight+ watches MTG Arena's log file, turns messy raw text into structured match data, saves that data locally, and can send cleaned summaries to Google Sheets.

The Big Picture

At a global level, the project is an automation pipeline:

```
MTGA Player.log
-> file tailer
-> line buffer
-> router
-> parsers
-> typed events
-> match state / summaries
-> local logs + Google Sheets + tier sync helpers
```

Here is the same idea in plain English:

- local JSONL files - Google Sheets through a webhook - tier-source snapshots for your workbook helper table

- 1 MTG Arena writes a giant running text file called `Player.log`.
- 2 Manasight+ watches that file for new lines.
- 3 It groups those lines into meaningful chunks.
- 4 It asks, "What kind of thing is this chunk?"
- 5 Specialized parser functions turn those chunks into structured event objects.
- 6 The app layer decides which events matter, remembers match context, and builds summaries.
- 7 The results get written to:

Why The Project Is Organized This Way

This project separates responsibilities.

That is one of the most important ideas in programming.

Instead of building one giant script that does everything, Manasight+ splits the work into layers:

- `log/` handles raw text coming off disk.

- `parsers/` know how to recognize specific MTGA message types.
- `events.py` defines the structured event shapes.
- `stream.py` moves events through the system.
- `app/` turns raw events into match summaries and sheet-ready rows.
- `tools/` contains user-facing helpers like the launcher and Scryfall tool.
- `tools/google_apps_script/Code.gs` is the Google Sheets side of the pipeline.

This makes the code easier to read, test, and repair.

A Very Short Computer Science Primer

Script, Module, Package

- A **script** is a file you run directly.
- A **module** is a Python file that contains reusable code.
- A **package** is a folder of modules that Python can import together.

In this project:

- `main.py` is a script.
- `src/manasight_parser_py/app/runner.py` is a module.
- `src/manasight_parser_py/` is a package.

Function and Method

- A **function** is a named block of reusable behavior.
- A **method** is a function attached to a class.

Examples:

- `include_event(event)` is a function.
- `summary.to_match_log_row()` is a method.

Class and Object

- A **class** is a blueprint.
- An **object** is a real instance built from that blueprint.

Example:

- `MatchSummary` is a class.
- `summary = MatchSummary(match_id="...")` creates an object.

Dictionary and List

- A **dictionary** stores named values like `{ "match_id": "...", "game_wins": 2 }`.

- A **list** stores ordered items like ["card1", "card2", "card3"].

MTGA payloads are mostly dictionaries.

JSON

JSON is a text format for structured data. It looks a lot like Python dictionaries and lists. MTGA logs often contain JSON inside bigger blocks of text. Google Sheets webhook payloads also use JSON.

State

State means "what the program currently remembers."

For example:

- current match ID
- current game number
- current player team
- mulligan count so far
- whether a match summary has already been posted

This project keeps a lot of live state in `state.py`.

Event-Driven Design

An **event** is "something that happened."

Examples:

- match started
- mulligan clicked
- sideboarding entered
- rank snapshot received
- game ended

Manasight+ is event-driven because the log keeps producing events over time, and the program reacts to them as they arrive.

Async

Async code is code designed for things that wait around a lot.

This project waits for:

- file updates
- new log entries
- webhook calls

`asyncio` lets the program keep listening without freezing.

The Folder Structure In Plain English

The main start button for the parser.

An old filename kept around so older shortcuts still work.

A separate start button for refreshing workbook tier data.

The real Python package.

Low-level file reading and log entry grouping.

Functions that recognize specific MTGA message shapes.

The high-level "business logic" of the project.

The desktop launcher UI.

The Google Sheets webhook script.

A separate utility for downloading Scryfall Oracle card data.

Output, snapshots, and other generated files.

Safety checks for the code.

Older versions and historical reference scripts.

- `main.py`
- `live_print_filtered_v11_match_summary.py`
- `sync_tier_buckets.py`
- `src/manasight_parser_py/`
- `src/manasight_parser_py/log/`
- `src/manasight_parser_py/parsers/`
- `src/manasight_parser_py/app/`
- `tools/auto_launcher/`
- `tools/google_apps_script/`
- `tools/scryfall_parser/`
- `data/`
- `tests/`
- `examples/`

The System Walkthrough

If you launch the parser and then play a match, this is what happens:

- 1 `main.py` starts the app.
- 2 `runner.py` performs startup checks.
- 3 `stream.py` opens `Player.log` and watches it from the end.

- 4 `tailer.py` reads new text from the file.
- 5 `entry.py` groups raw lines into log entries.
- 6 `router.py` asks the parser modules whether they recognize each entry.
- 7 A parser returns structured events such as `GameStateEvent` or `ClientActionEvent`.
- 8 `transforms.py` decides which events are worth keeping.
- 9 `state.py` updates the match summary in memory.
- 10 `outputs.py` writes local JSONL files and optionally POSTs rows to Google Sheets.
- 11 `Code.gs` receives those rows and writes them into the workbook.

That full chain is the heart of the project.

Script-By-Script Guide

Root Entry Scripts

`main.py`

Global job: start the main parser in the simplest possible way.

This file does almost no business logic. It mainly:

- figures out where `src/` lives
- makes sure Python can import the package
- imports the real app runner
- starts the async main loop

There are no local functions in this file. Its main purpose is to be easy to run.

`live_print_filtered_v11_match_summary.py`

Global job: act as a compatibility wrapper for an older filename.

This file exists so old shortcuts or habits still work. It simply imports `run_parser` from `main.py` and starts it.

`sync_tier_buckets.py`

Global job: refresh tier-source data without running the full match parser.

This is the "just update my meta deck buckets" script. It loads the same package path setup as `main.py`, then calls `sync_tier_sources`.

Package Root

`src/manasight_parser_py/__init__.py`

Global job: make the package pleasant to import.

This file re-exports important names like `MtgaEventStream`, `StreamError`, and the event classes so another script can say `from manasight_parser_py import MtgaEventStream`.

`src/manasight_parser_py/app/__init__.py`

Global job: expose the app runner without forcing a circular import.

It defines one small wrapper:

Imports `runner.main` only when called, then returns it. This lazy import keeps the package boundary cleaner.

- `main()`

Event Definitions

`src/manasight_parser_py/events.py`

Global job: define the vocabulary of the system.

This file says what kinds of structured events exist.

Important classes:

A category that says how urgently an event matters.

Stores the timestamp, raw bytes, and a SHA-256 hash of the raw bytes.

The common shape shared by all event classes.

Specific event types used by the rest of the app.

- `PerformanceClass`
- `EventMetadata`
- `BaseEvent`
- `GameStateEvent`, `ClientActionEvent`, `MatchStateEvent`, `GameResultEvent`, `RankEvent`, and others

Important method:

Automatically computes `raw_bytes_hash` after the object is created.

- `EventMetadata.__post_init__(self)`

Event Bus

`src/manasight_parser_py/event_bus.py`

Global job: let producers and consumers pass events around safely.

This is the internal message channel of the app.

Classes and methods:

A simple wrapper around an async queue.

Waits for the next event. Returns `None` when the bus is shutting down.

Creates the event bus and queue list.

Convenience constructor that uses the default queue size.

Creates and returns a new subscriber.

Sends one event to every active subscriber.

Sends a sentinel value so subscribers know the stream has ended.

- `Subscriber`
- `Subscriber.recv(self)`
- `EventBus.__init__(self, capacity)`
- `EventBus.with_default_capacity(cls)`
- `EventBus.subscribe(self)`
- `EventBus.publish(self, event)`
- `EventBus.close(self)`

Log Reading Layer

`src/manasight_parser_py/log/entry.py`

Global job: turn raw text lines into higher-level log entries.

Classes and functions:

Enum describing what kind of log header a line has.

A small object with two fields: header and body.

Starts an empty buffer.

Takes raw text, splits it into lines, groups lines into entries, and returns finished entries.

Forces the current partial entry to become a final entry.

Looks at one line and decides which header type it belongs to.

- `EntryHeader`
- `LogEntry`
- `LineBuffer.__init__(self)`
- `LineBuffer.feed(self, text)`
- `LineBuffer.flush(self)`
- `classify_line_header(line)`

`src/manasight_parser_py/log/tailler.py`

Global job: watch the log file as it changes.

Classes and methods:

Custom exception for file-tail problems.

A tiny container for entries plus a rotated flag.

Stores the path and initializes offsets and counters.

Returns how long the tailer has gone without seeing structured MTGA headers.

Opens a file and reads from byte 0.

Opens a file and starts at the end so old history is skipped.

Waits briefly, reads any newly added text, handles file rotation, and returns a TailBatch.

- TailerError
- TailBatch
- FileTailer.__init__(self, path)
- FileTailer.seconds_without_structured_headers(self)
- FileTailer.open_from_start(cls, path)
- FileTailer.open_from_end(cls, path)
- FileTailer.poll(self)

Router Layer

src/manasight_parser_py/router.py

Global job: decide which parser should handle a log entry.

Classes and functions:

Keeps simple counts such as how many entries were routed successfully.

Creates a router with empty stats.

Returns the current stats object.

Clears the stats.

Extracts a timestamp, sends the entry through parser functions, updates stats, and returns structured events.

Pulls a timestamp from the first line of a log entry.

Tries the parser modules in order and returns the first successful result.

- RouterStats
- Router.__init__(self)
- Router.stats(self)
- Router.reset(self)
- Router.route(self, entry)
- extract_timestamp(body)
- dispatch_to_parsers(entry, timestamp)

Stream Layer

`src/manasight_parser_py/stream.py`

Global job: connect the tailer, router, and event bus into one live pipeline.

Classes and methods:

High-level stream startup error.

Opens the tailer, creates the event bus, starts the background pipeline task, and returns the stream plus a subscriber.

Stops the stream and waits for the pipeline task to finish.

- `StreamError`
- `MtgaEventStream.start(cls, log_path)`
- `MtgaEventStream.shutdown(self)`

Important behavior:

- the stream now starts from the end of the file, not the beginning
- malformed routed entries get written to the bad-entry diagnostics file instead of silently killing the session

Utility and Privacy Helpers

`src/manasight_parser_py/util.py`

Global job: hold a few small generic helpers.

Gzip-compresses a log string.

Returns a SHA-256 hash of some bytes.

- `compress_log(text)`
- `content_hash(content)`

`src/manasight_parser_py/sanitize.py`

Global job: redact sensitive information from logs.

Replaces tokens, account IDs, display names, and local usernames with redacted placeholders.

- `scrub_raw_log(text)`

Parser Helper Module

`src/manasight_parser_py/parsers/api_common.py`

Global job: provide shared helper functions for parser modules.

Scans a larger text blob and returns the first JSON object or array it can decode.

Pulls structured JSON out of a log body and returns it as a dictionary.

Checks whether a body looks like a specific API request.

Checks whether a body looks like a specific API response.

Converts a mixed list into a clean list of integers.

- `find_json_value(text)`
- `parse_json_from_body(body, context="")`
- `is_api_request(body, name)`
- `is_api_response(body, name)`
- `normalize_int_list(value)`

Specific Parser Modules

`src/manasight_parser_py/parsers/metadata.py`

Global job: detect whether detailed logging is enabled.

Returns a `DetailedLoggingStatusEvent` when the entry says detailed logs are enabled or disabled.

- `try_parse(entry, timestamp)`

`src/manasight_parser_py/parsers/gre.py`

Global job: parse GRE game-state traffic, which is some of the most important match data.

Detects GRE game-state messages and can emit both `GameStateEvent` and `GameResultEvent`.

Builds a clean payload dictionary for a game-state event.

Checks whether the GRE state says the game has ended.

Builds a payload that captures who won, what the result type was, and why.

- `try_parse(entry, timestamp)`
- `build_game_state_payload(message, gsm)`
- `is_game_over(gsm)`
- `build_game_result_payload(gsm)`

`src/manasight_parser_py/parsers/client_actions.py`

Global job: parse actions the client sends back to the game engine.

Recognizes client-to-GRE messages and turns them into `ClientActionEvents`.

Pulls the real message payload out of the outer wrapper.

Builds a normalized mulligan payload.

Builds a normalized "pick N options" payload.

Builds a normalized submitted-deck payload.

- `try_parse(entry, timestamp)`
- `extract_inner_payload(parsed)`
- `build_mulligan(inner, envelope)`
- `build_select_n(inner, envelope)`
- `build_submit_deck(inner, envelope)`

`src/manasight_parser_py/parsers/match_state.py`

Global job: parse the overall match-room state.

Recognizes match-room state updates and creates a `MatchStateEvent`.

Normalizes the room state into match ID, players, result list, and match-complete information.

- `try_parse(entry, timestamp)`
- `build_payload(state_event)`

`src/manasight_parser_py/parsers/rank.py`

Global job: parse rank snapshots.

Recognizes the combined-rank API response and builds a `RankEvent`.

- `try_parse(entry, timestamp)`

`src/manasight_parser_py/parsers/session.py`

Global job: parse login/session lifecycle information.

Detects account updates, authentication responses, and logout events, then returns a `SessionEvent`.

- `try_parse(entry, timestamp)`

`src/manasight_parser_py/parsers/event_lifecycle.py`

Global job: parse event-queue lifecycle markers like entering pairing or claiming prizes.

Matches known event-lifecycle patterns and returns an `EventLifecycleEvent`.

- `try_parse(entry, timestamp)`

`src/manasight_parser_py/parsers/collection.py`

Global job: parse collection snapshots from `StartHook`.

Returns a `CollectionEvent` if the body contains `PlayerCards`.

- `try_parse(entry, timestamp)`

`src/manasight_parser_py/parsers/inventory.py`

Global job: parse inventory snapshots from `StartHook`.

Returns an `InventoryEvent` if the body contains `InventoryInfo`.

- `try_parse(entry, timestamp)`

App Layer

`src/manasight_parser_py/app/config.py`

Global job: hold configuration and environment-driven paths/flags.

This file does not define functions. It mainly defines constants such as:

- which event types should be kept
- where `Player . log` lives
- where local outputs should be stored
- whether raw rows, match-log rows, or tier sync are enabled

`src/manasight_parser_py/app/diagnostics.py`

Global job: make troubleshooting real instead of guesswork.

Configures a shared logger that writes to both the console and a daily runtime log file.

Returns the shared logger or a namespaced child logger.

Returns the path to the active runtime log file.

Saves a failed webhook POST to a daily JSONL file.

Saves a failed event-processing attempt to a JSONL file.

Saves a failed routing/parsing attempt to a JSONL file.

Internal helper that appends one diagnostics record to a daily file.

Converts a date into the project's daily folder format.

Converts objects into JSON-safe values for diagnostics storage.

- `setup_runtime_logging()`
- `get_logger(name=None)`
- `current_runtime_log_path()`
- `record_failed_post(row, exc, response_text="")`
- `record_event_failure(event, exc, stage)`
- `record_router_failure(entry, exc)`
- `_append_jsonl_record(root, prefix, record)`
- `_daily_folder_name(dt)`
- `_safe_json_value(value)`

`src/manasight_parser_py/app/extractors.py`

Global job: pull important facts out of messy payload dictionaries.

Returns the local player safely, even if the expected index is missing.

Pulls out match ID, game number, turn number, active player, phase, step, and stage from a game-state payload.

Searches several possible keys to find which player chose to start.

Normalizes scope values like MatchScope_Game into plain labels like Game.

Pulls out the identity of a game result: match, game, winner, result type, and reason.

Checks whether a result payload also includes a match-level result.

Returns a safe datetime for an event.

Returns a safe ISO timestamp string for an event.

- `_safe_local_player(players)`
- `_extract_turn_info(payload)`
- `_extract_starting_player_from_client_action(payload)`
- `_infer_scope_label(scope_value)`
- `_extract_game_result_identity(payload, context)`
- `_has_match_scope_result(payload)`
- `_event_datetime(event)`
- `_safe_iso(event)`

`src/manasight_parser_py/app/models.py`

Global job: define the clean in-memory summary objects.

GameSummary

Returns Play or Draw from the local player's perspective.

Returns W or L for the local player.

Says whether this game has enough data to be worth exporting.

Builds a human-readable dictionary of the game.

Converts one game into a sheet-ready row.

- `play_draw(self, player_team)`
- `result_for_player(self, player_team)`
- `has_summary_data(self)`
- `to_debug_dict(self, player_team)`
- `to_sheet_row(self, match)`

`_default_games()`

- Creates three empty GameSummary objects for games 1, 2, and 3.

MatchSummary

Updates first/last seen timestamps.

Returns the GameSummary for a given game number.

Stores who won a game.

Stores who was on the play for a game.

Stores mulligan count for a game.

Returns the opponent's team ID.

Uses explicit data if present; otherwise infers the next game's starter from the previous game result.

Returns Play or Draw for a specific game.

Counts how many games the local player won.

Counts how many games the local player lost.

Returns overall match result as W or L.

Adds mulligans across games.

Counts completed games.

Returns 1 for a win, 0 for a loss, or blank if unknown.

Returns game win rate as a fraction.

Says whether the summary is complete enough to export.

Converts detailed rank information into a workbook-friendly label like Mythic %.

Returns the match date.

Builds a verbose debugging view of the whole match.

Converts the match into a MatchSummary row for Sheets.

Converts the match into the workbook-shaped MatchLogRow.

Exports one row per game when game-level data exists.

- touch(self, timestamp)
- game(self, game_number)
- set_game_winner(self, game_number, winner_team)
- set_game_starting_player(self, game_number, starting_player)
- set_game_mulligans(self, game_number, mulligans)
- opponent_team(self)
- effective_starting_player(self, game_number)
- game_play_draw(self, game_number)
- game_wins(self)
- game_losses(self)
- match_wl(self)
- total_mulligans(self)

- `total_games(self)`
- `match_win_flag(self)`
- `game_win_rate(self)`
- `is_ready(self)`
- `rank_bucket(self)`
- `played_date(self)`
- `to_debug_dict(self)`
- `to_sheet_row(self)`
- `to_match_log_row(self)`
- `to_game_sheet_rows(self)`

`src/manasight_parser_py/app/state.py`

Global job: hold the parser's live memory and build summaries over time.

This is one of the most important files in the project.

Builds a stable key for de-duplication.

Tracks how many mulligans have happened for a specific game.

Creates a MatchSummary if one does not already exist.

Updates the summary's first and last timestamps.

The heart of the state system. Reads incoming events and updates the live match summary.

Small helper that asks whether a summary is ready to export.

Returns a sheet row if the match summary is ready.

Returns per-game summary rows.

Returns the workbook-shaped match log row.

- `_context_key(match_id, game_number)`
- `_next_mulligan_count(match_id, game_number, decision)`
- `_ensure_match_summary(match_id)`
- `_set_first_last(summary, event)`
- `_update_match_summary(event)`
- `_match_summary_ready(summary)`
- `build_match_summary_row(match_id)`
- `build_game_summary_rows(match_id)`
- `build_match_log_row(match_id)`

`src/manasight_parser_py/app/transforms.py`

Global job: decide what to keep and how to format it.

Builds the common base shape for raw event rows sent to Sheets.

Decides whether an event is important enough to keep at all.

Converts an event into a JSON-safe dictionary for local JSONL storage.

Converts one raw event into zero, one, or multiple raw sheet rows.

Builds one short console line describing an event.

- `_base_sheet_row(event)`
- `include_event(event)`
- `to_serializable(event)`
- `to_sheet_rows(event)`
- `summarize(event)`

`src/manasight_parser_py/app/outputs.py`

Global job: write data out of the app.

Sends one row to the webhook. Returns True on success and writes failed rows to disk on failure.

Builds the day-folder name like 04_18_26.

Finds or creates the JSONL file path for a given day.

Appends one serialized row to the local daily JSONL log.

- `post_row_to_google_sheets(row)`
- `_daily_folder_name(event_dt)`
- `_ensure_daily_log_path(event_dt)`
- `append_local_jsonl(local_row, event_dt)`

`src/manasight_parser_py/app/tier_sync.py`

Global job: refresh tier-source deck data from outside websites.

Data classes

Converts one tier-source record into a row dictionary.

Returns the snapshot's rows.

Counts the rows that are usable deck rows.

Counts how many sources successfully produced deck rows.

Builds a one-line status summary.

- `TierRecord.to_sheet_row(self)`
- `TierSyncResult.records(self)`
- `TierSyncResult.ok_rows(self)`

- `TierSyncResult.available_sources(self)`
- `TierSyncResult.summary_line(self)`

Functions

Runs the full tier refresh, writes a local snapshot, and optionally posts it to the webhook.

Builds the full workbook-ready snapshot payload.

Saves the snapshot JSON locally.

Loads the deck-name alias table from disk.

Scrapes MTGGoldfish and returns tier records.

Scrapes MTGTop8 and returns tier records.

Tries to scrape Untapped and records an unavailable status if the real rows are not public.

Builds a successful tier record.

Builds a placeholder record that explains why a source is unavailable.

Maps raw site names to your preferred canonical deck names.

Converts percentages or letter tiers into Tier 1, Tier 2, Tier 3, or Fringe.

Pulls a percent number out of a text string.

Trims and normalizes whitespace.

Creates a case-insensitive key for alias matching.

Writes JSON to disk.

- `sync_tier_sources(post_to_webhook=True)`
- `build_tier_snapshot_payload()`
- `write_tier_snapshot(snapshot)`
- `load_normalization_overrides()`
- `_scrape_mtggoldfish(session, refreshed_at, overrides)`
- `_scrape_mtgtop8(session, refreshed_at, overrides)`
- `_scrape_untapped(session, refreshed_at, overrides)`
- `_build_ok_record(source_key, refreshed_at, raw_name, meta_share_pct, tier_letter, overrides)`
- `_build_unavailable_record(source_key, refreshed_at, notes)`
- `normalize_archetype_name(source_key, raw_name, overrides)`
- `classify_tier_bucket(meta_share_pct, tier_letter)`
- `_parse_percent(text)`
- `_clean_text(text)`
- `_normalize_key(text)`
- `_write_json(path, payload)`

`src/manasight_parser_py/app/runner.py`

Global job: run the whole application.

This is the real operational center of the parser.

Performs startup checks such as log-path existence, webhook validity, and tier-normalization JSON readability.

Initializes diagnostics, runs preflight checks, optionally refreshes tier buckets, starts the stream, processes events forever, writes outputs, and captures event-level failures.

- `_startup_issues()`
- `main()`

Command-Line Tool

`src/manasight_parser_py/bin/scrub.py`

Global job: expose the log scrubber as a command-line program.

Reads text from standard input, scrubs it, writes the cleaned version, and returns an exit code.

- `main()`

User-Facing Tools

`tools/auto_launcher/manasight_launcher_auto.py`

Global job: give you a simple desktop UI for running the parser.

Top-level helpers:

Loads saved launcher settings from JSON.

Writes launcher settings to JSON.

Checks whether MTGA is currently running.

- `load_settings()`
- `save_settings(data)`
- `is_process_running(process_name)`

LauncherApp methods:

Creates the window, state variables, and background timers.

Builds the whole Tkinter interface.

Lets the user choose a project folder.

Lets the user choose which script to run.

Lets the user choose `Player.log`.

Scans the project folder for runnable parser scripts.

Saves the current UI settings.

Checks that the chosen paths exist and nothing is already running.

Returns whether any sheet-posting mode is turned on.

Validates settings, warns about blank webhooks, sets environment variables, and launches the parser subprocess.

Reads live output from the running subprocess.

Moves queued log lines into the launcher text box.

Monitors whether MTGA opens or closes.

Asks whether to start the parser when MTGA is detected.

Appends one line to the launcher log window.

Stops the running subprocess.

Opens the match-log output folder in the file explorer.

Saves settings and handles shutdown behavior when the window closes.

- `__init__(self)`
- `_build_ui(self)`
- `browse_project_root(self)`
- `browse_script_file(self)`
- `browse_player_log(self)`
- `refresh_scripts(self)`
- `persist_settings(self)`
- `_validate(self)`
- `_sheet_posting_enabled(self)`
- `start_script(self)`
- `_reader_thread(self)`
- `_drain_log_queue(self)`
- `_watch_mtga_loop(self)`
- `_prompt_start_for_mtga_session(self)`
- `_log(self, message)`
- `stop_script(self)`
- `open_output_folder(self)`
- `on_close(self)`

`tools/scryfall_parser/scryfall_parser.py`

Global job: download Scryfall Oracle data and filter it by format legality.

Downloads the latest Oracle bulk data file into data/oracle_data/.

Filters the Oracle data by format legality and writes both JSON and CSV outputs.

Prompts the user for a format, downloads the file, and filters it.

- `download_latest_oracle_file()`
- `filter_legal_cards(oracle_path, fmt_key)`
- `main()`

Google Sheets Support Script

tools/google_apps_script/Code.gs

Global job: receive webhook posts and update the workbook.

Think of this file as the "Google Sheets half" of Manasight+.

Core functions:

Adds the custom MTGA Tools menu to the workbook.

Opens the target spreadsheet by ID.

Returns a sheet by name, or null.

Returns a sheet by name, or throws an error if it is missing.

Builds a dictionary from header names to column numbers.

Special case of `getHeaderMap_` for the Match Log tab.

Finds the last used row in a specific column.

Chooses the next row where a new Match Log entry should go.

Main webhook entry point. Reads JSON, decides what kind of payload it is, writes it to the right place, and returns a JSON response.

Converts a MatchSummary payload into a summary-sheet row.

Converts a raw event payload into an archive row.

Ensures the match has a row in Match Log and updates it.

Maps parser payload fields to workbook columns.

Writes mapped values into a Match Log row.

Fills default values like OK or Yes where needed.

Writes a value only if the target cell is blank.

Returns the first non-blank value.

Returns the first numeric value.

Adds wins and losses.

Computes game win percentage.

Converts W or L into 1 or 0.

Converts a value into a Google Sheets date if possible.

Converts booleans or truthy text into Yes / No.

Builds the workbook-style rank label.

Converts winning team into W or L for the local player.

Returns JSON from the web app.

Appends a row to Webhook Debug.

- `onOpen()`
- `getWorkbook_()`
- `getSheet_(ss, name)`
- `getSheetOrThrow_(ss, name)`
- `getHeaderMap_(sheet, headerRow)`
- `getMatchLogHeaderMap_(sheet)`
- `findLastNonBlankRowInColumn_(sheet, col, startRow)`
- `getNextMatchLogAppendRow_(sheet)`
- `doPost(e)`
- `buildSummaryRow_(data)`
- `buildArchiveRow_(data)`
- `upsertMatchLogFromPayload_(ss, data)`
- `buildMatchLogFieldMap_(data)`
- `writeMatchLogFields_(sheet, row, fieldMap)`
- `applyMatchLogDefaults_(sheet, row, fieldMap)`
- `setIfBlank_(sheet, row, col, value)`
- `firstDefined_(...values)`
- `firstDefinedNumber_(...values)`
- `computeTotalGames_(gamesWon, gamesLost)`
- `computeGameWinRate_(gamesWon, totalGames)`
- `deriveMatchWinFlag_(matchWin)`
- `parseDateOrBlank_(value)`
- `boolToYesNo_(value)`
- `deriveMyRank_(data)`
- `deriveGameResult_(winnerTeam, playerTeam)`
- `jsonResponse_(payload)`
- `logDebug_(debugSheet, status, matchId, data, destination)`

Tier helper functions:

Sets up the hidden support tab and workbook controls for tier data.

Rewrites the tier-source data tab from a webhook snapshot.

Creates the hidden support tab if needed.

Replaces the support tab contents with fresh records.

Converts one tier record into a sheet row.

Builds the dropdown and formulas in Helper Table.

Extracts the distinct source labels.

Rebuilds the tier bucket formulas, including Other / Unknown.

Hides a support sheet if it is safe to do so.

- `prepareTierSourceHelpers()`
- `upsertTierSourceSnapshot_(ss, data)`
- `ensureTierSourceDataSheet_(ss)`
- `rewriteTierSourceData_(sheet, records)`
- `buildTierSourceDataRow_(record)`
- `configureHelperTableTierSourceControls_(helperSheet, tierSheet)`
- `getUniqueTierSourceLabels_(sheet, lastRow)`
- `resetTierBucketSpillArea_(helperSheet)`
- `hideSheetIfSafe_(ss, sheet)`

Workbook cleanup functions:

Hides old tabs and freezes parser-managed columns.

Replaces formulas with values in parser-managed columns.

Hides older helper/archive tabs.

Finds or creates the Match Log row for a match.

- `prepareWorkbookForPhase1()`
- `freezeParserManagedMatchLogColumns_(sheet)`
- `hideLegacyMtgaTabs_(ss)`
- `ensureMatchLogRowForMatchId_(ss, matchId)`

Test helpers:

Writes a fake Match Log row for testing.

Writes fake tier-source rows for testing.

- `testMatchLogWrite()`
- `testTierSourceWrite()`

Historical Example Scripts

`examples/live_print.py`

Global job: a minimal "just print live events" example.

Starts the stream and prints events as they arrive.

- `main()`

`examples/live_print_filtered_v8.py`

`examples/live_print_filtered_v9.py`

`examples/live_print_filtered_v10_fixed.py`

Global job: older all-in-one parser scripts from before the codebase was split into clean modules.**

These historical files combine many behaviors that are now separated into:

- `extractors.py`
- `state.py`
- `transforms.py`
- `outputs.py`
- `runner.py`

Their functions have the same broad jobs as the modern modular versions:

Old helper functions that later moved into `extractors.py`.

Old state-tracking helpers that later moved into `state.py`.

Older transformation and export helpers that later moved into `transforms.py`.

Older output helpers that later moved into `outputs.py`.

Old monolithic runtime loop that later became `runner.py`.

- `_safe_local_player, _extract_turn_info, _extract_starting_player_from_client_action`
- `_context_key, _next_mulligan_count, _update_context`
- `include_event, to_serializable, summarize, _base_sheet_row, to_sheet_rows`
- `post_row_to_google_sheets, _event_datetime, _daily_folder_name, _ensure_daily_log_path`
- `main()`

`v9.py` also contains `_build_game_result_rows`, which was an earlier attempt to create explicit game-result rows.

Test Files

The test suite is part of the project's learning story because it shows what the code is trying to guarantee.

`tests/test_app_config.py`

Verifies that config paths default to the repo's data/ folders.

- `test_default_match_logs_root_uses_data_folder(monkeypatch)`

`tests/test_app_extractors.py`

Verifies that starting-player extraction still works when the useful data is nested.

- `test_extract_starting_player_from_nested_raw_client_action()`

`tests/test_app_models.py`

Verifies MatchSummary export shape.

Verifies per-game export rows.

Verifies the workbook-shaped Match Log export.

Verifies the play/draw fallback rule.

Verifies that real data wins over inferred data.

- `test_match_summary_sheet_row_matches_workbook_shape()`
- `test_game_summary_rows_exist_for_future_exports()`
- `test_match_log_row_matches_first_phase_workbook_shape()`
- `test_missing_later_game_starting_player_is_inferred_from_previous_game_result()`
- `test_explicit_starting_player_beats_inferred_starting_player()`

`tests/test_app_outputs.py`

Test helper that simulates an HTTP failure.

Verifies that failed posts land in the diagnostics file.

- `_FakeHttpResponse.raise_for_status(self)`
- `test_failed_webhook_posts_are_saved_locally(tmp_path, monkeypatch)`

`tests/test_entrypoints.py`

Verifies that the legacy filename still points at the real parser.

- `test_legacy_entrypoint_delegates_to_modular_main()`

`tests/test_log_entry_headers.py`

Verifies that UTC-style numeric prefixes do not break header recognition.

- `test_classify_line_header_handles_utc_log_prefix()`

tests/test_match_summary_from_match_state.py

Verifies that final match-state result lists correctly build summaries.

- `test_match_state_final_results_populate_match_summary()`

tests/test_parsers.py

Verifies metadata parsing.

Verifies match-state parsing.

Verifies deck-submission parsing.

Verifies GRE game-over parsing.

- `test_metadata_parse()`
- `test_match_state_parse()`
- `test_client_submit_deck_parse()`
- `test_gre_game_over_emits_game_state_and_game_result()`

tests/test_sanitize.py

Verifies that the scrubber removes sensitive strings.

- `test_scrub_removes_bearer_and_user_path()`

tests/test_scryfall_tool.py

Verifies that the Scryfall tool writes into the repo, not somewhere external.

- `test_scryfall_tool_writes_to_repo_data_folder()`

tests/test_tailer.py

Verifies the fix that prevents stale startup rows.

- `test_open_from_end_skips_existing_log_history(tmp_path)`

tests/test_tier_sync.py

Test helper that stores fake HTML.

Fake success status method.

Fake requests session that stores canned HTML.

Returns the canned fake response.

Verifies percentage-based tier mapping.

Verifies letter-tier mapping.

Verifies alias overrides.

Verifies MTGGoldfish scraping.

Verifies MTGTop8 scraping.

Verifies the graceful Untapped fallback.

- `_FakeResponse.__init__(self, text)`
- `_FakeResponse.raise_for_status(self)`
- `_FakeSession.__init__(self, html)`
- `_FakeSession.get(self, url, timeout)`
- `test_classify_tier_bucket_supports_percent_thresholds()`
- `test_classify_tier_bucket_supports_letter_tiers()`
- `test_normalize_archetype_name_uses_source_specific_override()`
- `test_scrape_mtggoldfish_extracts_archetypes_and_meta_share()`
- `test_scrape_mtgtop8_extracts_breakdown_rows()`
- `test_scrape_untapped_reports_unavailable_when_public_html_is_premium_locked()`

How To Read This Project As A Beginner

If you want to learn coding from this project, I would read it in this order:

This shows how the program starts.

This shows the top-level workflow.

This shows what the program is trying to build.

This shows how the program remembers what is happening.

This shows how the program chooses what matters.

This shows how the program digs specific values out of messy data.

This shows how raw logs become structured events.

This shows how a user-facing GUI starts a subprocess.

This shows the Google Sheets half of the pipeline.

- 1 `main.py`
- 2 `app/runner.py`
- 3 `app/models.py`
- 4 `app/state.py`
- 5 `app/transforms.py`
- 6 `app/extractors.py`
- 7 `stream.py, router.py, parsers/`
- 8 `tools/auto_launcher/manasight_launcher_auto.py`

A Good Beginner Exercise

Try answering these questions by reading the code:

- 1 Where does the parser first learn the `match_id`?
- 2 Where does it decide a log entry is important enough to keep?
- 3 Where does it infer `Play` or `Draw`?
- 4 Where does it decide the row is ready to send to Sheets?
- 5 Where does it save failures now if the webhook breaks?

If you can answer those five questions, you are already reading real software, not just toy examples.

What To Be Proud Of

This is not a tiny beginner script anymore.

It has:

- a streaming pipeline
- modular parsers
- stateful summary building
- Google Sheets integration
- a GUI launcher
- web scraping helpers
- diagnostics
- tests

That is a serious personal software project.

Final Mental Model

If you forget everything else, remember this:

- `log/` reads
- `parsers/` recognize
- `events.py` names things
- `stream.py` moves things
- `state.py` remembers
- `models.py` defines the clean output
- `transforms.py` decides what to keep
- `outputs.py` writes

- `runner.py` coordinates
- `Code.gs` finishes the trip into Google Sheets

That is the whole project in one sentence.